

JavaScript — Modules (ES6 Modules)

 **Note** A complete deep-dive into the JavaScript module system — the difference between modules and scripts, strict mode, named and default exports, import syntax, and live bindings.

◆ Part 1 — Module vs Script

Before ES6 modules, every JavaScript file shared the **same global scope**. Variables from one file could accidentally overwrite variables from another. Modules solve this.

The Full Comparison

☰ Comparison Table

Feature	Module	Script
Scope	Module scope (isolated)	Global scope (shared)
Mode	Strict mode (automatic)	Sloppy mode (default)
<code>this</code> at top level	<code>undefined</code>	<code>window</code>
<code>import</code> / <code>export</code>	Yes	Cannot
HTML usage	<code><script type="module"></code>	<code><script></code>
Execution	Deferred (after HTML parsed)	Immediate (blocks HTML)
Downloaded	Once (cached)	Every time

How to Use in HTML

```
<!-- Script - old way, global scope -->
<script src="script.js"></script>
```

```
<!-- Module - new way, isolated scope -->
<script src="script.js" type="module"></script>
```

◆ Part 2 — Global Scope Pollution (The Problem)

Understanding why modules exist requires understanding the global scope problem first.

Implicit Global Variables

```
// In a regular script
fname = "mona"; // No var/let/const - automatically becomes
window.fname!
console.log(window.fname); // "mona"
```

```
var aName = "mona"; // var at top level → window.aName
console.log(window.aName); // "mona"

aName = "ali"; // window.aName is now "ali"
```

```
function test() {
  var aName = "mona"; // var inside function → function scope, NOT
  global
}
test();
console.log(window.aName); // undefined - stays in function scope
```

```
function test() {
  aName = "mona"; // NO var/let/const inside function → becomes
  global!
}
test();
console.log(window.aName); // "mona" - leaked to global scope!
```

 **Note** This accidental global leaking is one of the most dangerous behaviors in JavaScript. Modules eliminate it — variables are scoped to the module, never leak to `window`.

Part 3 — Strict Mode

`'use strict'` enables a stricter mode that catches common bugs:

```
'use strict'
```

```
fname = "mona"; // ReferenceError: fname is not defined
// Without strict mode: silently creates a global variable
// With strict mode: throws immediately
```

What Strict Mode Catches

- Assigning to undeclared variables → `ReferenceError`
- Deleting variables or functions → `SyntaxError`
- Duplicate parameter names → `SyntaxError`
- Using reserved words as variable names → `SyntaxError`
- `this` in regular functions = `undefined` instead of `window`

How to Enable

```
// Entire file – place at the very top
'use strict';

// Single function only
function test() {
  'use strict';
  // ...
}
```

 **Note** Modules automatically use strict mode — you don't need to write `'use strict'` in a module file. It's always on.

● Part 4 — Exports

Named Exports

Export specific values by name. A module can have **multiple** named exports.

Inline Export

```
// script2.js
export function add(a, b) {
  return a + b;
}

export function sub(a, b) {
  return a - b;
}

export function mul(a, b) {
  return a * b;
}

export let x = 10;
```

Export at the Bottom (Cleaner)

```
// script2.js
function add(a, b) { return a + b; }
function sub(a, b) { return a - b; }
function mul(a, b) { return a * b; }

// Export all at once – easier to see what's public at a glance
export { mul, sub, add };
```

Both syntaxes are equivalent. Exporting at the bottom is often preferred — it separates implementation from the public API.

Default Export

A module can have **only one** default export — it represents the "main thing" this module provides.

```
// Inline
export default function divide(a, b) {
  return a / b;
}

// Or separately
function divide(a, b) { return a / b; }
export default divide;
```

 **Note Key difference:** Named exports use the exact name. Default export has no name attached — the importer can name it anything.

Named + Default Together

A module can have both:

```
// script2.js
export function add(a, b) { return a + b; } // named
export function mul(a, b) { return a * b; } // named
export default function divide(a, b) { return a / b; } // default
```

Part 5 — Imports

Importing Named Exports

```
// script.js
import { add, sub, mul } from './script2.js';
```

```
console.log(add(2, 3)); // 5
console.log(sub(2, 3)); // -1
```

 **Note** The `./` is required for local files. Without it, JS looks in `node_modules`.

Renaming with `as`

```
import { add as myAdd, mul as myMul } from './script2.js';

console.log(myAdd(2, 3)); // 5
// console.log(add(2, 3)); // ReferenceError - 'add' doesn't exist,
// renamed to 'myAdd'
```

Renaming is useful when:

- Avoiding name conflicts with existing variables
- Giving shorter or more descriptive local names

Importing Default Export

```
import myDef from './script2.js'; // no curly braces for default

console.log(myDef(10, 2)); // 5
```

The name `myDef` is completely arbitrary — you choose the name because default exports have no enforced name.

Importing Both Default and Named

```
import myDef, { add as myAdd, mul as myMul } from './script2.js';
// default first, then named in curly braces
```

```
console.log(myDef(10, 2)); // 5 (divide)
console.log(myAdd(2, 3)); // 5 (add)
console.log(myMul(2, 3)); // 6 (mul)
```

Import Everything with `* as`

```
import * as all from './script2.js';

console.log(all.add(2, 3)); // 5
console.log(all.sub(2, 3)); // -1
// all.default - accesses the default export
```

This imports all named exports as properties of one object. Useful when a module has many exports and you don't want to list them all.

🟡 Part 6 — Live Bindings (Important!)

ES6 module imports are **live bindings** — they are a **live connection** to the exported value, not a copied snapshot. When the exported value changes, the imported value updates too.

```
// script2.js
export let x = 10;

setInterval(function() {
  x++; // incrementing x in script2
}, 1000);
```

```
// script.js
import { x } from './script2.js';

setInterval(function() {
  console.log(x); // 10, 11, 12, 13... - reflects live changes from
```

```
script2!  
, 1000);
```

🔗 **Note** This is different from CommonJS (`require`), where you get a copied value at the time of import. ES6 imports are always live. **Important constraint:** You cannot reassign a live binding from the importing side:

```
import { x } from './script2.js';  
x = 20; // SyntaxError - you can't reassign an import binding  
// Only the exporting module can change x
```

○ Part 7 — Module Scope Isolation

Each module has its own scope. Variables don't leak between modules:

```
// script2.js  
var x = 5;           // stays in script2 - NOT on window  
let y = 20;          // stays in script2
```

```
// script.js  
console.log(x); // ReferenceError - script2's x is not visible here
```

This is the core benefit of modules — no accidental global pollution. Each module controls exactly what it shares via `export`.

● Part 8 — Module Loading Behavior

Modules are Deferred

Module scripts are automatically deferred — they don't block HTML parsing:

```
<!-- This blocks HTML parsing until the script runs -->  
<script src="app.js"></script>
```

```
<!-- This does NOT block - HTML parses first, then module runs -->
<script type="module" src="app.js"></script>
```

Modules are Downloaded Once

If two modules both import from `utils.js`, the browser downloads and executes `utils.js` only once — the result is cached and shared.

Modules Execute in Order

```
// script.js
console.log("Script 1");
import * as all from './script2.js';
```

```
// script2.js
console.log("Script 2");
```

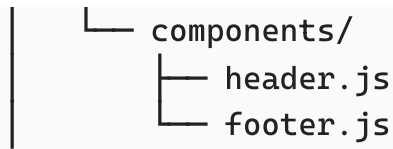
Output:

```
Script 2  ← imported modules run first
Script 1
```

 **Note Imports are hoisted** — even though the `import` statement appears after `console.log("Script 1")`, the module is loaded and executed first.

● Part 9 — Organizing a Real Module System

```
project/
├── index.html
├── js/
│   ├── main.js      (entry point - imports everything)
│   ├── utils.js     (pure utility functions)
│   └── api.js        (API calls)
```



```
// utils.js
export function formatDate(date) { ... }
export function truncate(str, len) { ... }
export default function debounce(fn, delay) { ... }

// api.js
import { formatDate } from './utils.js';
export async function getUser(id) { ... }
export async function getCountry(name) { ... }

// main.js
import { formatDate, truncate } from './utils.js';
import debounce from './utils.js';
import { getUser, getCountry } from './api.js';
```

Complete Summary

Module vs Script:

Module: isolated scope, strict mode, this=undefined, import/export
Script: global scope, sloppy mode, this=window, no modules

Strict Mode ('use strict'):

- Prevents implicit globals
- Catches more errors at runtime
- Automatic in modules

Named Exports:

```
export function name() {}      - inline
export { name1, name2 }       - at bottom (preferred)
Import: import { name } from './file.js'
Rename: import { name as alias } from './file.js'
```

Default Export:

```
export default function() {} - one per module, no name required
```

Import: import anything from './file.js' - you choose the name

Import All:

import * as obj from './file.js' - all named exports as properties

Live Bindings:

Imports are live - reflect changes from the exporting module

Cannot reassign from importing side

Module Loading:

Deferred (non-blocking)

Cached (downloaded once)

Imported modules execute before the importing module

Abdullah Ali

Contact : +201012613453